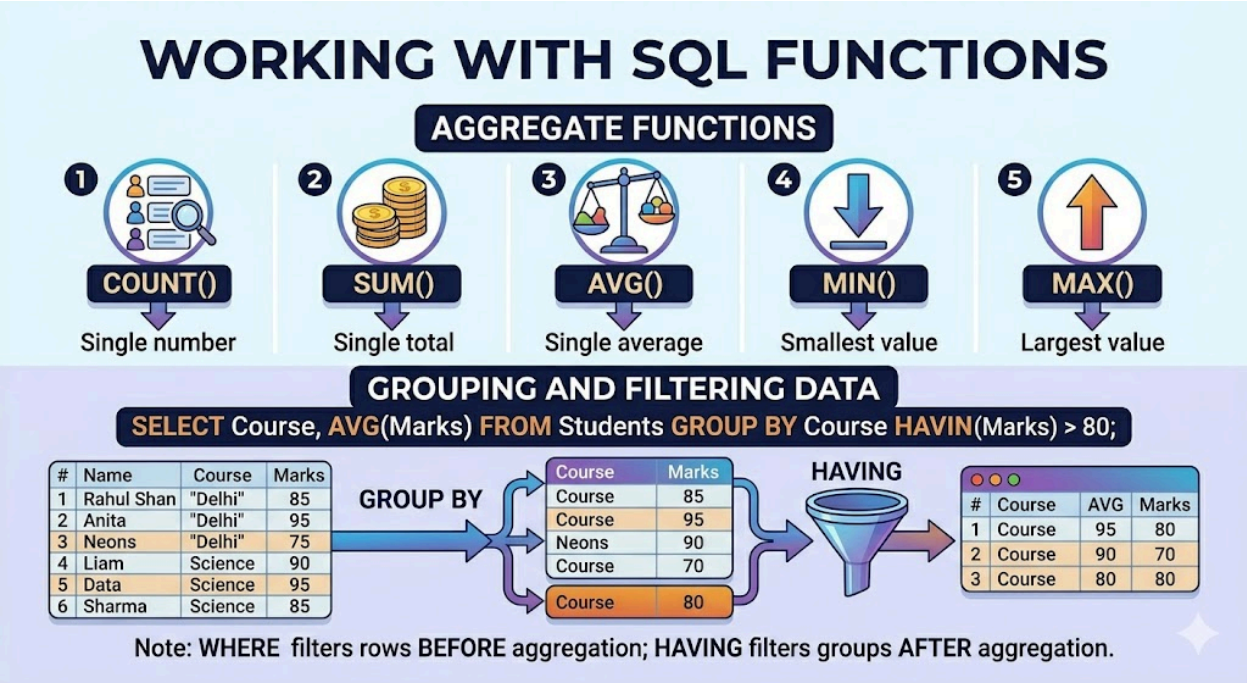
Day 2: Aggregate, Joins

Welcome back to Day 2 of the SQL Bootcamp! Today, we are leveling up our data manipulation skills. We will move beyond basic data retrieval and look at how to perform calculations, safely modify our database, and connect data across multiple tables.



🧮 1. Working with Functions

In SQL, functions allow us to perform calculations on our data to extract meaningful insights.

Aggregate Functions

These functions perform a calculation on a set of values and return a single summarizing value. The most common aggregate functions include:

- `COUNT()`: Returns the number of rows.
- `SUM()`: Adds up all values in a numeric column.
- `AVG()`: Calculates the average value.

- `MIN()`: Finds the lowest value.
- `MAX()`: Finds the highest value.

Grouping and Filtering Data

Often, you want to apply these functions to specific categories rather than the whole table.

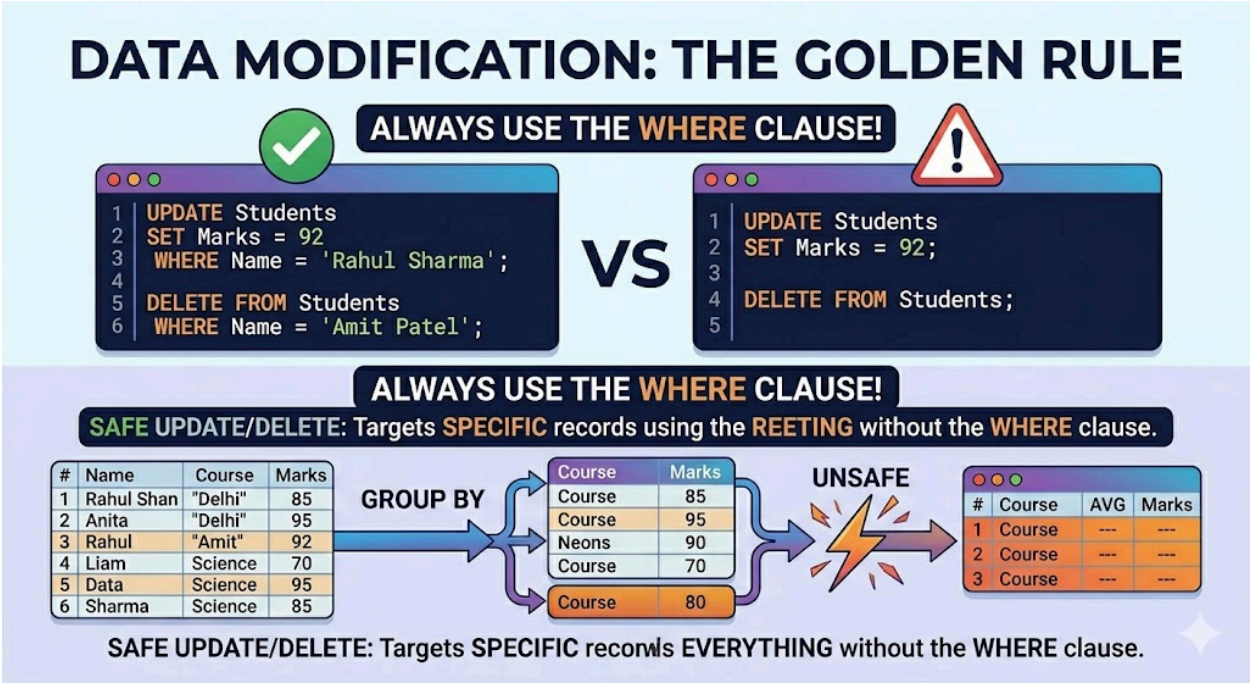
`GROUP BY`: We use this clause to group data with identical values into summary rows.

`HAVING`: We use this to filter groups. (Note: You cannot use `WHERE` to filter aggregate functions, which is why `HAVING` exists!).

Example: Let's find the average marks per course.

SQL

```
SELECT Course, AVG(Marks) AS Average_Marks
FROM Students
GROUP BY Course;
```



⚠️ 2. Updating and Deleting Data

Data changes over time. Students change courses, correct their names, or leave the program.

`UPDATE`: Use this to modify existing records in your table.

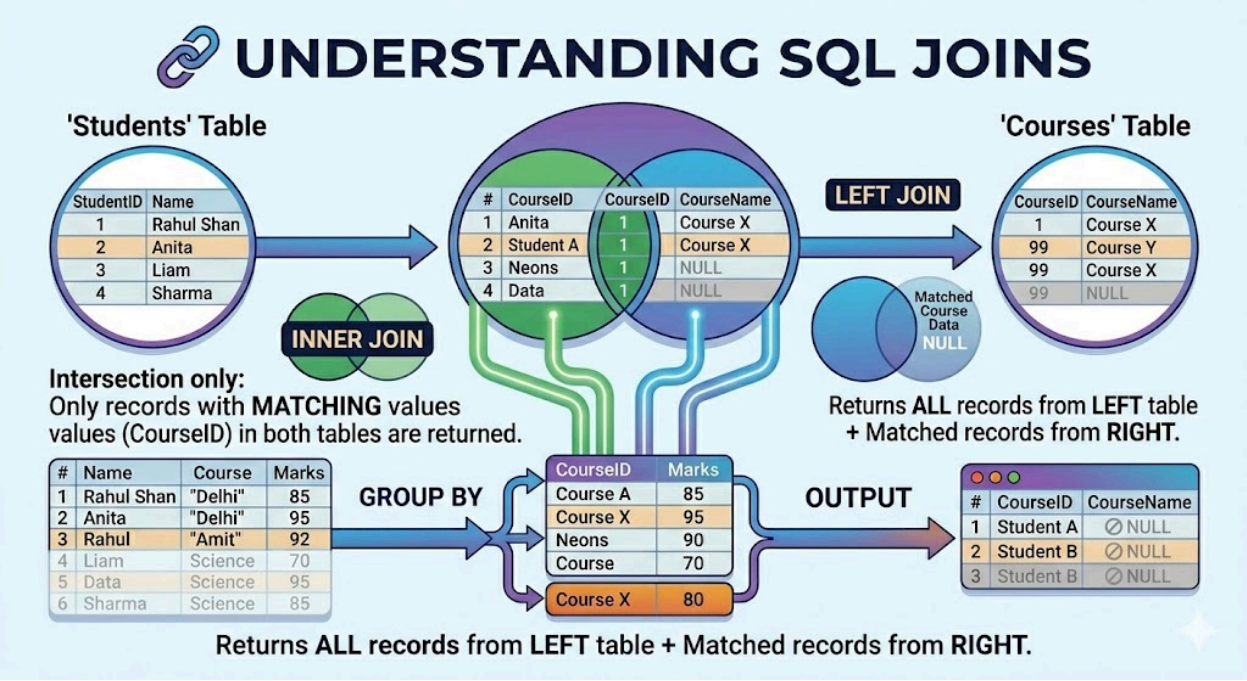
`DELETE`: Use this to permanently remove records.

The Golden Rule: Always use the `WHERE` clause! The importance of the `WHERE` clause cannot be overstated. If you write an `UPDATE` or `DELETE` statement without a `WHERE` clause, you will modify or erase **every single record** in your table.

Examples:

SQL

- ```
  - Safely updating a specific student's marks
  UPDATE Students
  SET Marks = 92WHERE Name = 'Rahul Sharma';
  - Safely deleting a specific student recordDELETE FROM Students
  WHERE Name = 'Amit Patel';
  ```



🔗 3. Joins

In real-world databases, information is rarely kept in just one massive table. Instead, data is normalized (split) into multiple tables.

Why Joins are Used: Joins are used for combining related tables based on a common column between them.

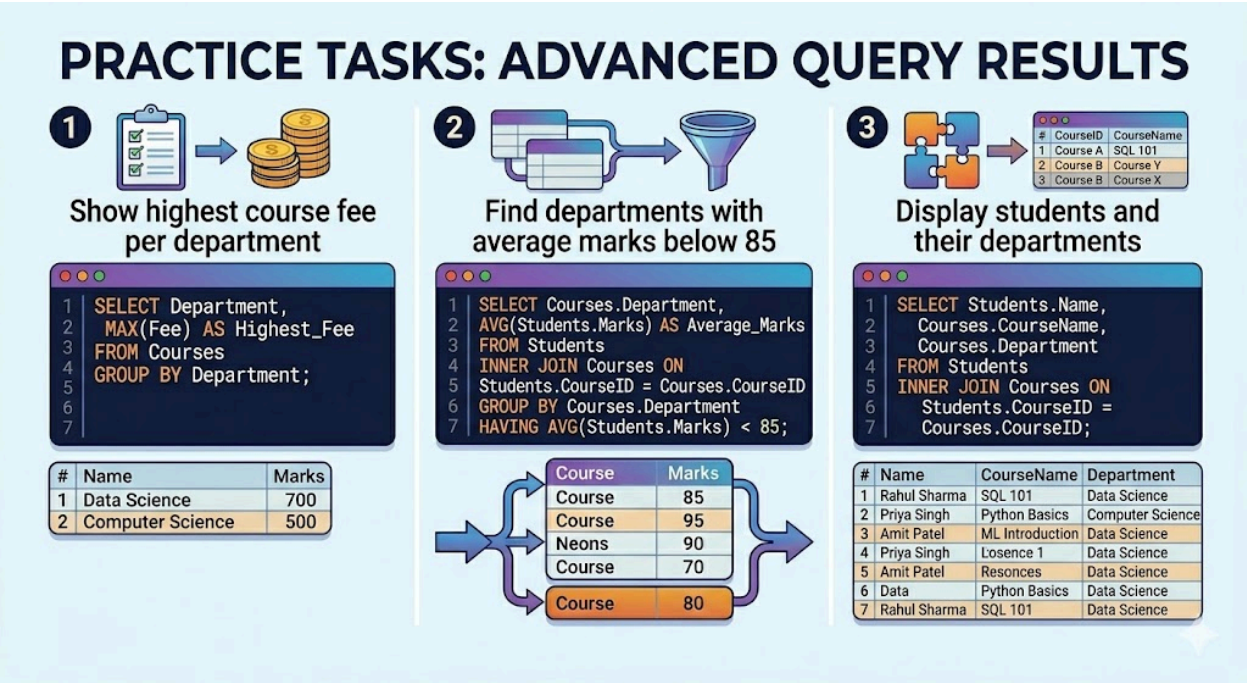
We will focus on two primary types of joins:
  ```

- `INNER JOIN`: Returns records that have matching values in *both* tables.
- `LEFT JOIN`: Returns *all* records from the left table, and the matched records from the right table. (If there is no match, the result is NULL on the right side).

Example: Let's link the `Students` and `Courses` tables together using a common `CourseID`.

SQL

```
SELECT Students.Name, Courses.CourseName
FROM Students
INNER JOIN Courses ON Students.CourseID = Courses.CourseID;
```



💻 Practice Tasks

It's time to put these concepts together. For these tasks, you first need to create the `Students` and `Courses` tables and write queries to join and summarize data.

Setup code (run this in your SQL environment first):

SQL

```
CREATE TABLE Courses (
    CourseID INT PRIMARY KEY,
    Department VARCHAR(50),
    CourseName VARCHAR(50),
    Fee INT
```

```
);
```

```
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    CourseID INT,
    Marks INT
);
-- (Assume tables are populated with sample data)
```

Task 1: Write a query to show the highest course fee per department.

SQL

```
SELECT Department, MAX(Fee) AS Highest_Fee
FROM Courses
GROUP BY Department;
```

Task 2: Write a query to find departments with average marks below 85.

SQL

```
SELECT Courses.Department, AVG(Students.Marks) AS Average_Marks
FROM Students
INNER JOIN Courses ON Students.CourseID = Courses.CourseID
GROUP BY Courses.Department
HAVING AVG(Students.Marks) < 85;
```

Task 3: Use a `JOIN` to display students and the courses offered in their department.

SQL

```
SELECT Students.Name, Courses.CourseName, Courses.Department
FROM Students
INNER JOIN Courses ON Students.CourseID = Courses.CourseID;
```